

LAB 10

COUNTERS & CLOCK DIVIDER

CONTENTS

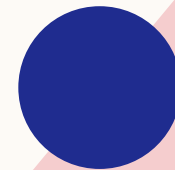
Behavioral Modeling

Clock signal

Counter

Clock divider

TopLevel



TYPES OF CIRCUITS

3

<u>Features</u>	<u>Combinational Circuit</u>	<u>Sequential Circuit</u>
<u>Definition</u>	Output depends only on the current inputs.	Output depends on both current inputs and past inputs (stored state).
<u>Memory</u>	No memory element.	Contains memory elements (flip-flops, latches) to store past inputs.
<u>Clock Dependency</u>	Not dependent on a clock signal.	Requires a clock signal for synchronization.
<u>Behavior</u>	Instantaneous response based on inputs.	Changes state only at specific clock edges.
<u>Example Component</u>	Logic gates (AND, OR, XOR, etc.), Multiplexers, Decoders, Adders.	Flip-Flops, Registers, Counters, Finite State Machines (FSMs).

BEHAVIORAL MODELING

- It is primarily used to model sequential circuits, but can also be used to model pure combinatorial circuits.
- The behavioral statements – structured procedural statements – for modeling the behavior of a design are:
 1. **initial** Statements
 2. **always** Statements
- These statements cannot be nested.

INITIAL STATEMENT

- Command: **initial**
- This starts at time 0, executes ***exactly once*** during a simulation.
- In case there are multiple *initial* blocks, each block starts to execute concurrently at time '0', and finishes execution independently of other blocks.
- Used for initialization, monitoring waveforms etc. – settling sequence of values in simulation (testbenches)
- Initial block is not synthesizable in design modules i.e. cannot be converted into a hardware schematic with digital elements.

INITIAL STATEMENT

```
initial
m = 1'b0;    //Single Statement; doesn't need to be grouped
initial
begin
#5 = 1'b1;
#25 = 1'b0;  //Multiple Statement; need to be grouped
end
```

ALWAYS STATEMENT

- Command: **always**
- This starts at time 0, executes like a while loop, it continuously keeps running repetitively.
- Needs a stopping condition.
- Syntax:
`always @ (enable) // when enable is high the statement under
this block executes
begin
....
end`

ALWAYS STATEMENT

```
// Code your design here
module clock_gen (clock);
output clock;
reg clock;
//Initialize clock at time zero
initial
clock = 1'b0;
always
//Toggle clock every half-cycle (time period = 20) always
#10 clock = ~clock;
initial
#1000 $finish;
endmodule
```

PROCEDURAL ASSIGNMENT

- Updating the values of variables.
- The value placed remains unchanged until another procedural assignment updates the variable.
- Types:
 - 1. Blocking Assignment**
 - 2. Non-blocking Assignment**

BLOCKING ASSIGNMENT

- Executes immediately in order (sequentially).
- Updates the variable immediately before executing the next line.
- Operator '=' is used.

```
x=0; y=1;      //Scalar assignments  
count = 0;     //Assignment to integer variables  
count = count + 1; // Assignment to an integer (increment)
```

NON- BLOCKING ASSIGNMENT

- Executes all statements in parallel – scheduling of the assignments without blocking execution of the statements that follows.
- Operator ' $\leftarrow$ ' is used

```
reg x,y  
reg (15:0) reg_a, reg_b;  
integer count,z;  
initial  
begin  
x = 0; y=1; z= 0; // blocking  
count = 1; // blocking  
count <= count + 1; // non-blocking  
z <= count + y; // non-blocking  
end
```

These statements are executed simultaneously.

DELAY BASED TIMING CONTROL

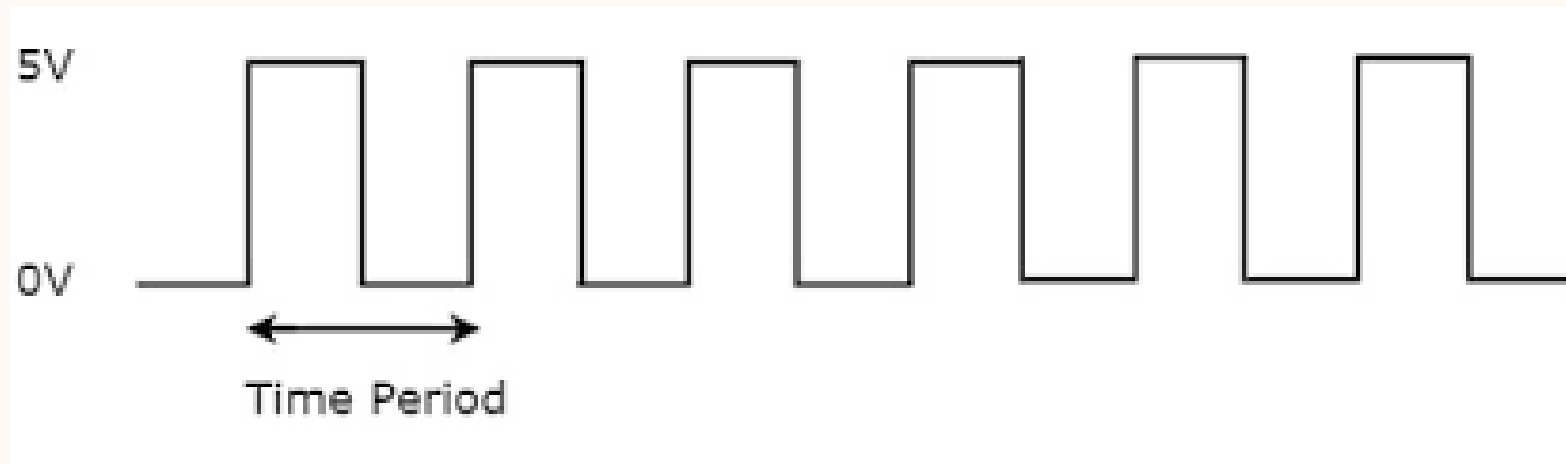
- Specifies the time duration between when the statement is encountered and when it is executed.
- Symbol: '#'

```
reg x, y;  
initial  
begin  
x = 0;           // no delay control  
#10 y = 1;      // delay execution of y = 1 by 10 units  
end
```

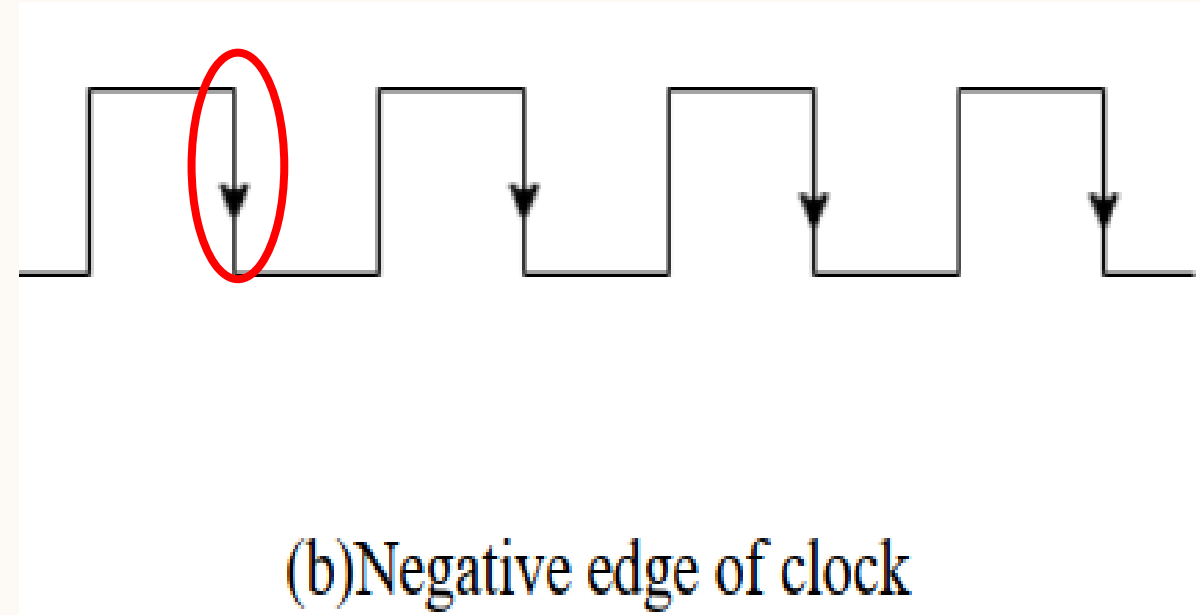
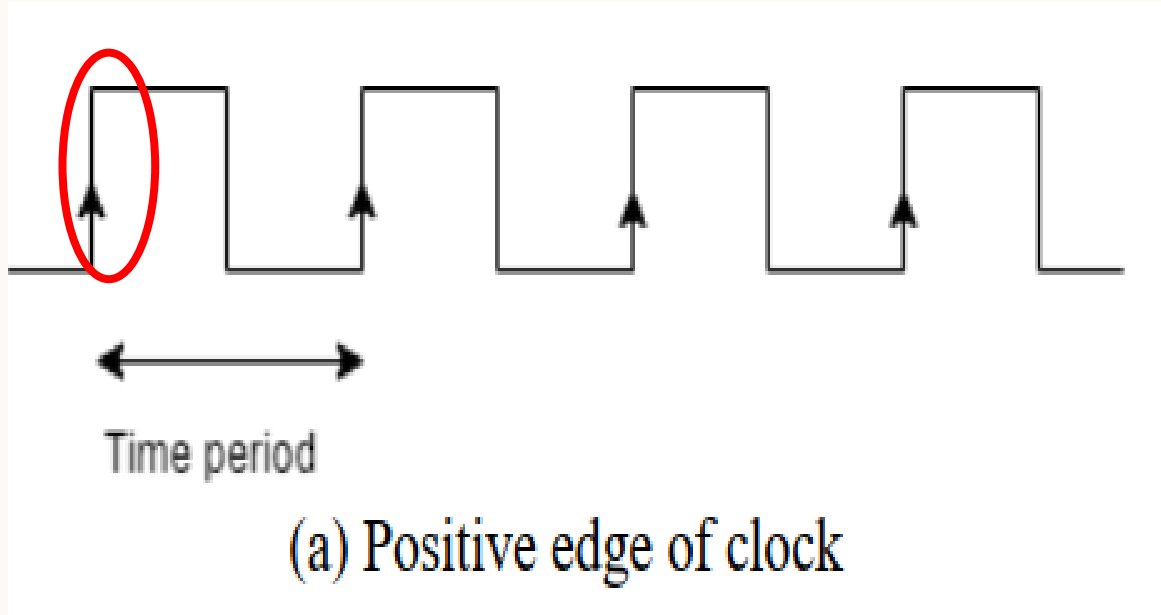
SYNCHRONOUS CIRCUITS

- A synchronous circuit is a digital circuit in which the changes in the state of memory elements are synchronized by a clock signal.

CLOCK SIGNAL



EVENT BASED TIMING CONTROL



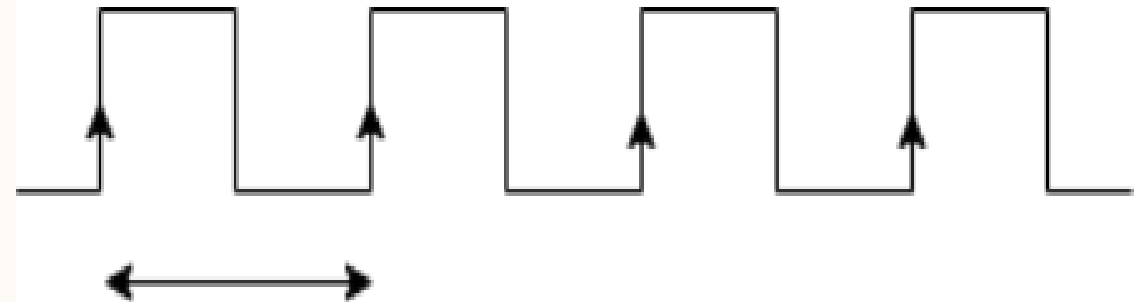
EVENT BASED TIMING CONTROL

```
initial a=0;
```

```
always @ (posedge clk)
```

```
a=a+1; //increment
```

Variable 'a' is incremented only when the clock hits positive edge of the cycle.



Time period

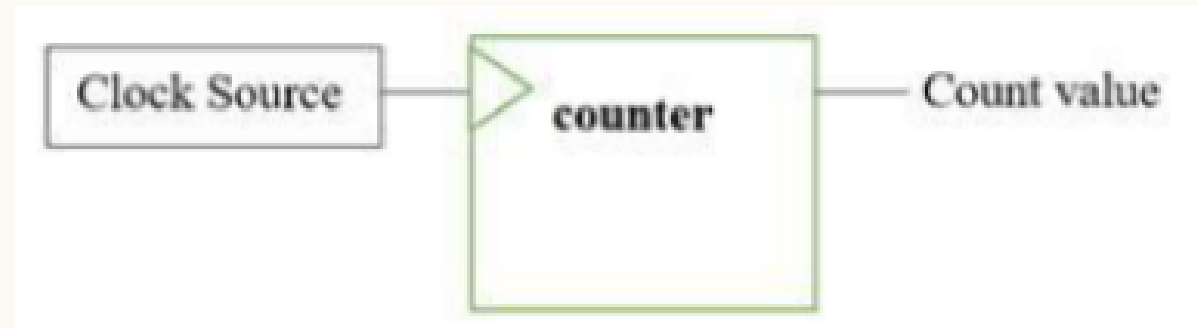
(a) Positive edge of clock

EVENT BASED TIMING CONTROL

```
@(clock) q = d;           //q =d is executed whenever signal clock changes value
@(posedge clock) q=d      //q =d is executed whenever signal clock does a positive
transition @(negedge clock) q=d    //q =d is executed whenever signal clock does a
negative transition always @(posedge clock) //always block is executed at positive edge
of clock
begin
....
end
```

WHAT IS A COUNTER?

- A counter is a sequential circuit that increments its count value upon each positive edge of the clock pulse.
- An *n -bit* counter counts up to $2^n - 1$ count value.



IMPLEMENTATION OF 4-BIT COUNTER

For each positive edge of clock pulse, the count registers increments until its value is 15. Once the count value exceeds 15 the count value is reset to 0.

```
module counter_4bit(clk,count);
    input clk;
    output [3:0] count;
    reg [3:0] count;

    initial
        count = 0;
    always @ (posedge clk)
        begin
            if (count <= 15)
                begin
                    count <= count + 1;
                end
            else
                begin
                    count <= 0;
                end
        end
    end
endmodule
```

Design Module Code

```
module tb();
    reg clk;
    wire [3:0] count;
    counter_4bit c1(.clk(clk),.count(count));

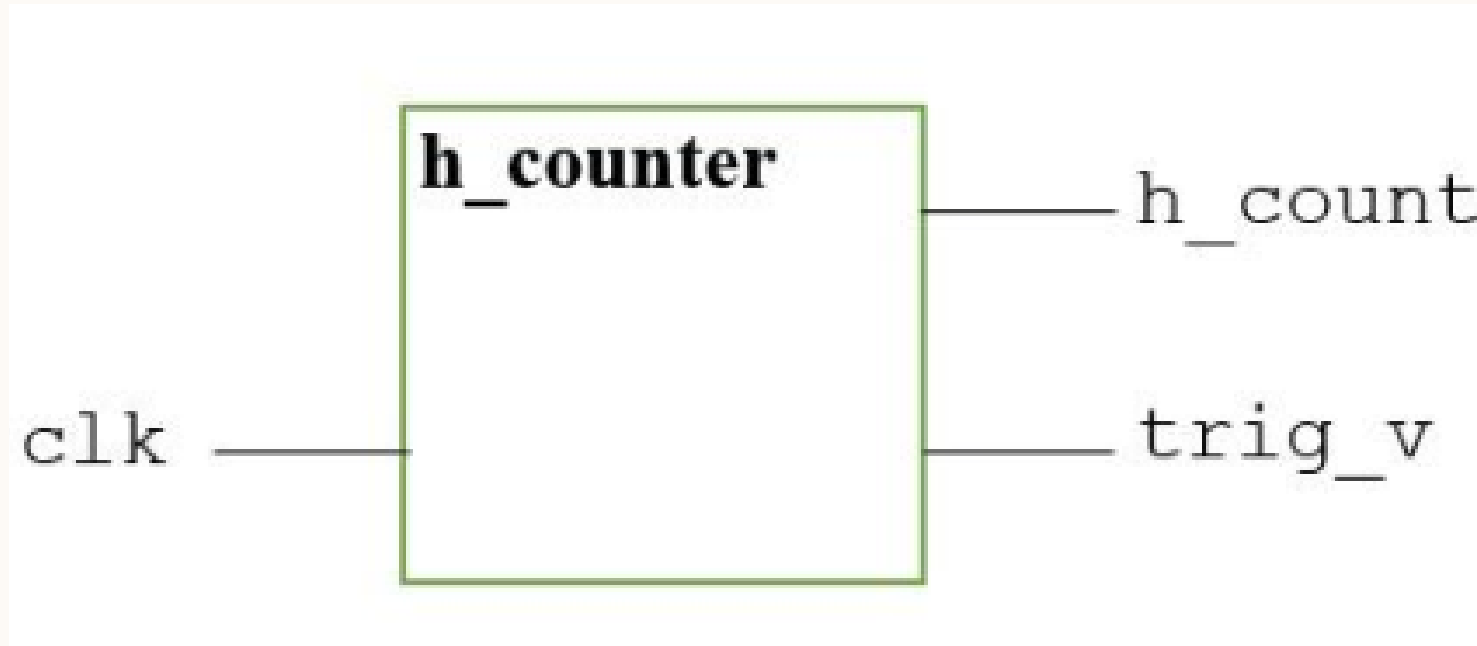
    initial
        clk = 1'b0;

    always
        #5 clk = ~clk;

    initial
        #200 $finish;
endmodule
```

Testbench Code

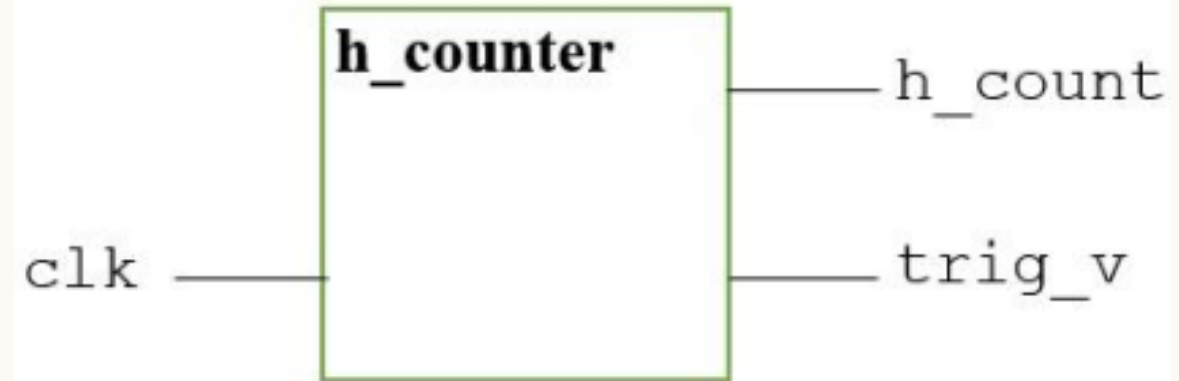
TASK A - h_counter



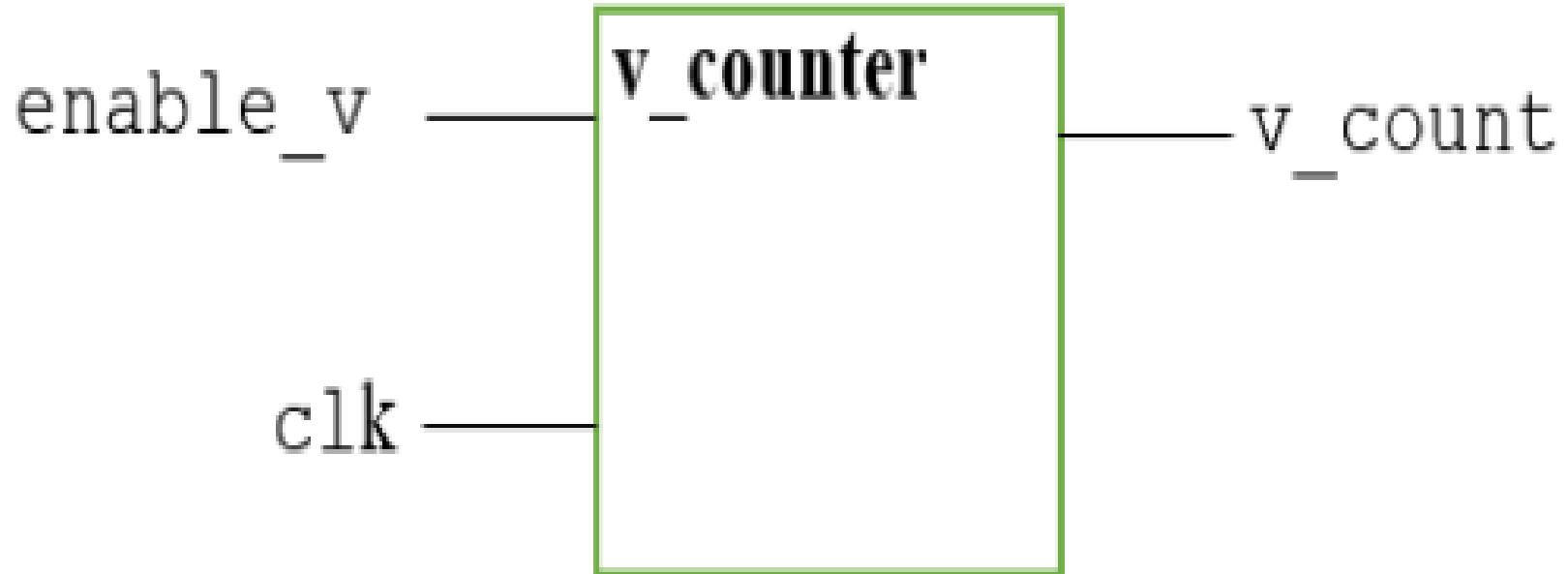
TASK A

20

- 10-bit counter
- Increment the counter output (h_count) at every positive edge of clock signal (clk) and counts up to 799.
- The counter resets to 0 if the value exceeds 799.
- trig_v is set to 1 when counter resets (counter has reached to its maximum value of 799).
- trig_v would be zero if count is greater than 1 after reset (count is less than 799).

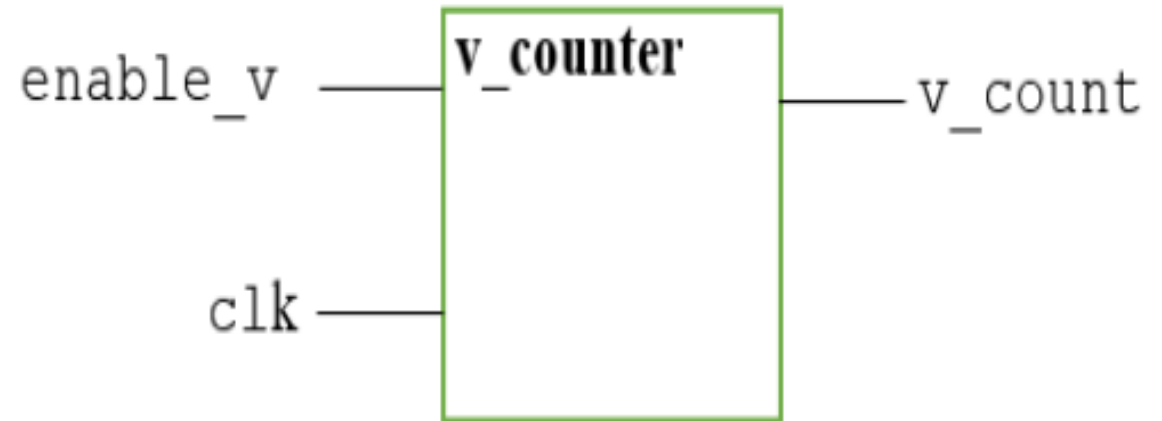


TASK B - v_counter



TASK B

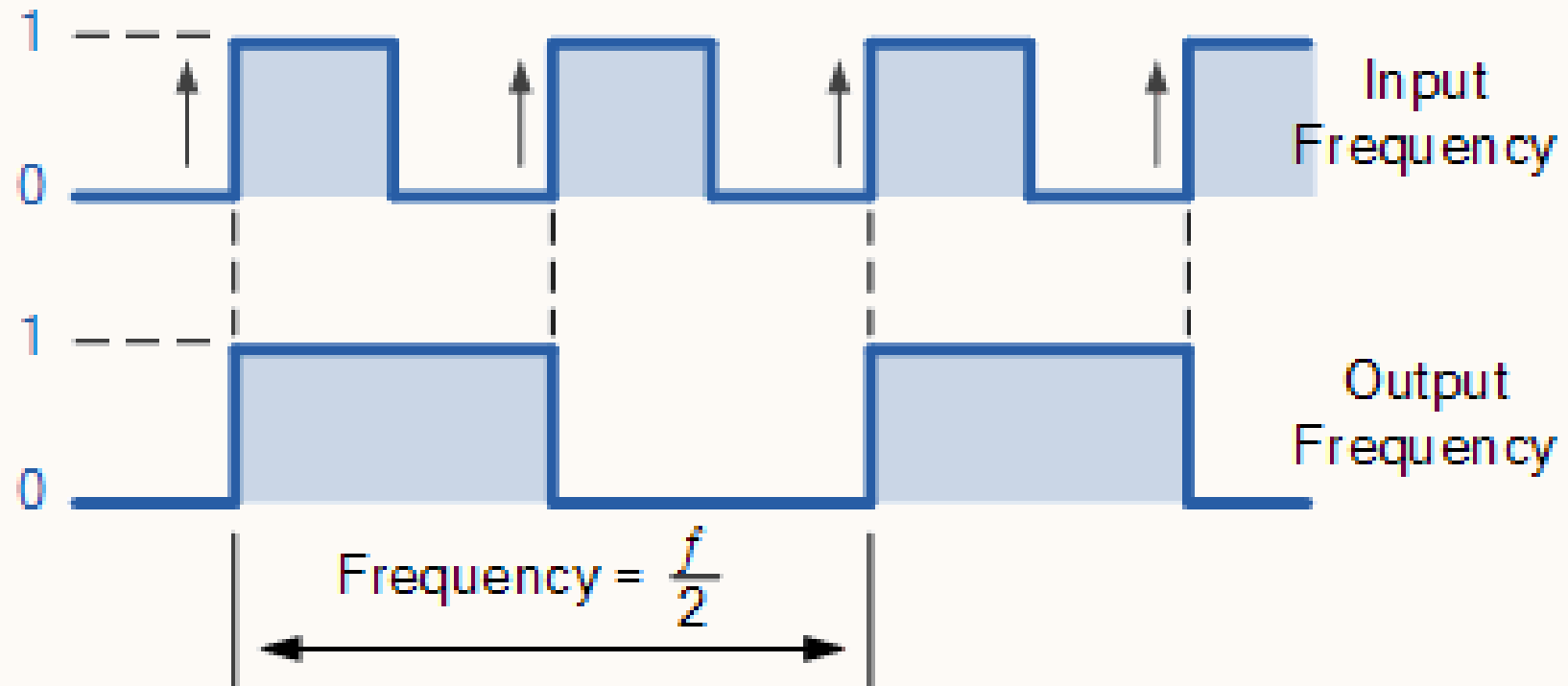
- 10-bit counter
- Increment the counter output (v_count) at every positive edge of clock signal (clk) and counts up to 524, only if enable_v signal is 1.
- For testbench:
At first keep the input variable 'enable' = 0, and after some delay (delay must be bigger than the delay for switching of clock edge), keep the enable as 1.



WHAT IF I WANT TO SLOW DOWN THE SPEED OF MY CLOCK?

- By default, Basys-3 board's frequency is 100MHz.
- To drive the VGA output, requirement of the clock frequency is 25MHz.

CLOCK DIVIDER



IMPLEMENTATION OF CLOCK DIVIDER

24

- Clock divider module has one input *clk*, and one output *clk_d*.
- In this experiment, the *clk* input will take the default clock (of 100 MHz).
- A register is defined with the name *count*, which counts to a value of *div_val* = 1. The value of one-bit output *clk_d* is toggled every time when the counter becomes equal to *div_val*.

$$\text{div_value} = \frac{\text{input clock frequency}}{2 \times \text{desired clock frequency}} - 1$$

- In Verilog, the constant value can be defined using the keyword - *parameter*.
- Here input clock frequency is 100MHz and desired frequency is 25MHz.
- So using this formula, we found the value of *div_value* is 1.

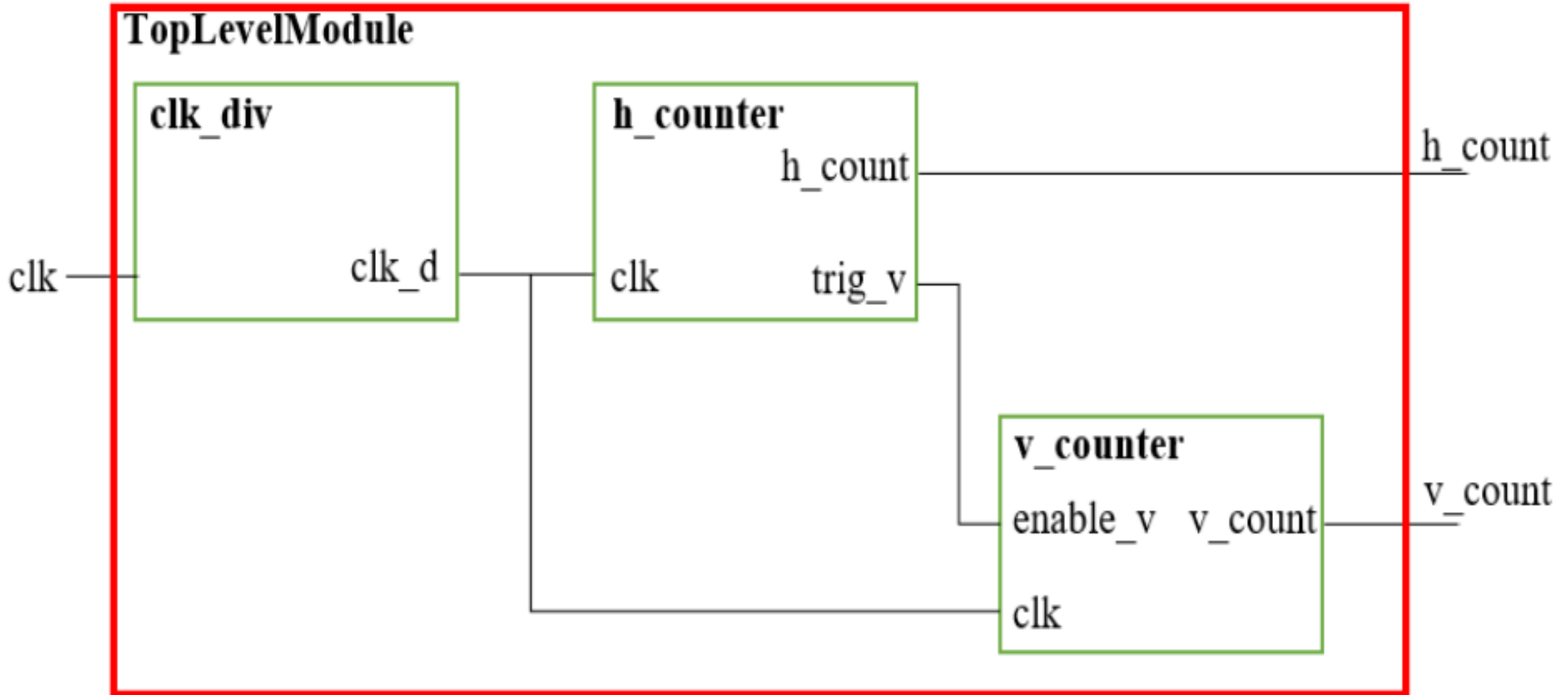
```
module clk_div (clk, clk_d);
    parameter div_value = 1;
    input clk;
    output reg clk_d;
    reg count;
    initial
        begin
            clk_d = 0;
            count = 0;
        end
    always @(posedge clk)
        begin
            if (count == div_value)
                count <= 0; // reset count
            else
                count <= count + 1; // count
        end
    up
    always @(posedge clk)
        begin
            if (count == div_value)
                clk_d <= ~ clk_d; //toggle
            end
        end
endmodule
```

TASK C

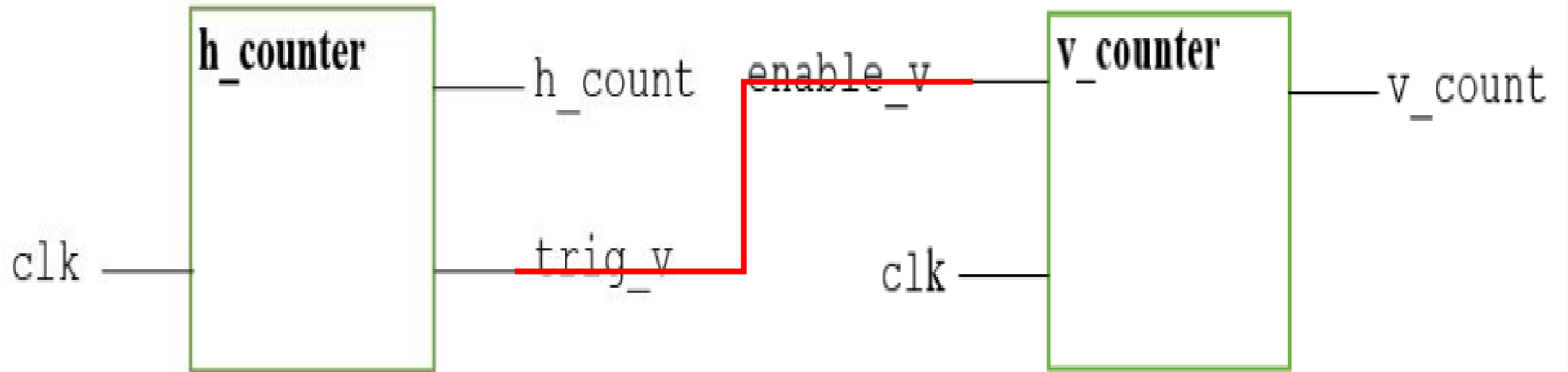
- Develop the testbench to test the design module code provided under Task (c).
- Hint: Initialize clock at 0, and toggle with such a delay so that its time period is 2units.

EXERCISE – TOPLEVEL MODULE

27

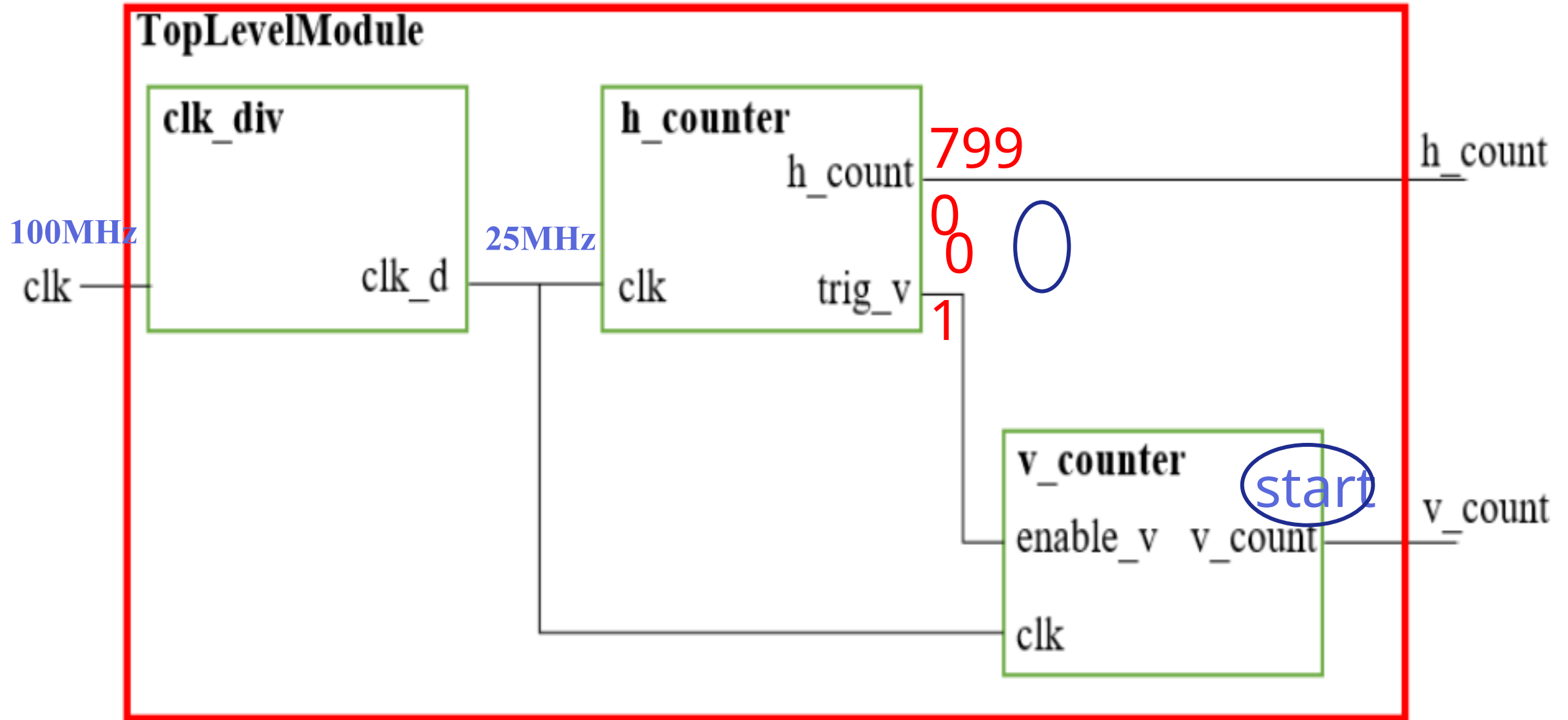


REMEMBER!



EXERCISE – TOPLEVEL MODULE

29



SIMULATION - SCREENSHOT REQUIRED IN REPORT FOR TASK C

